CUDA HyperLogLog

Mithun

The Problem

How do you count unique visitors to a website with billions of users?

Cardinality estimation problem

Naive Approach: Store every single user ID in memory

1 billion users × 8 bytes = 8 GB RAM

The Solution: HyperLogLog - Trade perfect accuracy for efficiency

1 billion users ≈ 5 KB RAM (98% accuracy)

Why this Matters?

Real-World Applications

YouTube: Daily view counting across billions of videos

Google Analytics: Unique visitor tracking

Network Traffic Analysis: Unique IP address counting

The Challenge: Need real-time processing at a massive scale

How does HyperLogLog work

192.168.1.5 --- Hash Function ---> 00011110... 3 leading zeros

192.24.0.1 --- Hash Function ---> 0101110... 1 leading zero

134.84.8.8 --- Hash Function ---> 1011000... 0 leading zeros

Max leading Zeros Seen = 3

Our intuition: More leading zeros = rarer pattern = likely saw more unique items

Probablity = $\frac{1}{2^3} = \frac{1}{8}$

We have likely seen 8 unique IP Addresses

Why use Multiple Buckets?

Single Bucket = One unlucky hash ruins your estimate

Multiple Buckets = Better Accuracy

Why Harmonic mean and not regular mean

A regular mean gives too much weight to rare, large estimates.

The harmonic mean limits their influence across buckets.

HLL combines these estimates with a bias-correction factor.

Raw estimate: alpha(m) x m^2 / sum(2^(-M[j]))

Source: Flajolet et al. (2007), HyperLogLog, doi:10.46298/dmtcs.3545

Hash function and Bucket count

Hash Function:

Transforms input into random-looking binary numbers

- MurmurHash - Industry Standard and Fast
- CityHash - Google's fast hash optimized for x86-64
- FarmHash - Google's successor to CityHash

Bucket Count:

$$m = 2^b$$

Higher b -> more buckets -> lower error

But more memory + more GPU work

Goal

Implement HyperLogLog using CUDA for parallel processing

Determine the optimal configuration (hash function and bucket count)

Dataset: BMS-POS

Items: 515598

Transactions: 3367020

Pseudo code for parallel update process

```
Kernel Launch: ProcessItems<<<grid_size, block_size>>>(items, registers, b)

__global__ Kernel ProcessItems(items, registers, b):
    // Each thread processes one item
    thread_id ← blockIdx.x × blockDim.x + threadIdx.x

    If thread_id < n:
        hash_value ← Hash(items[thread_id])

        // Get bucket index from lower b bits
        bucket ← hash_value AND ((1 << b) - 1)

        // Count leading zeros in remaining bits
        remaining_bits ← hash_value >> b
        leading_zeros ← CountLeadingZeros(remaining_bits) + 1

        // Atomic update to handle concurrent writes
        AtomicMax(registers[bucket], leading_zeros)

    Return registers
```

Experiment A

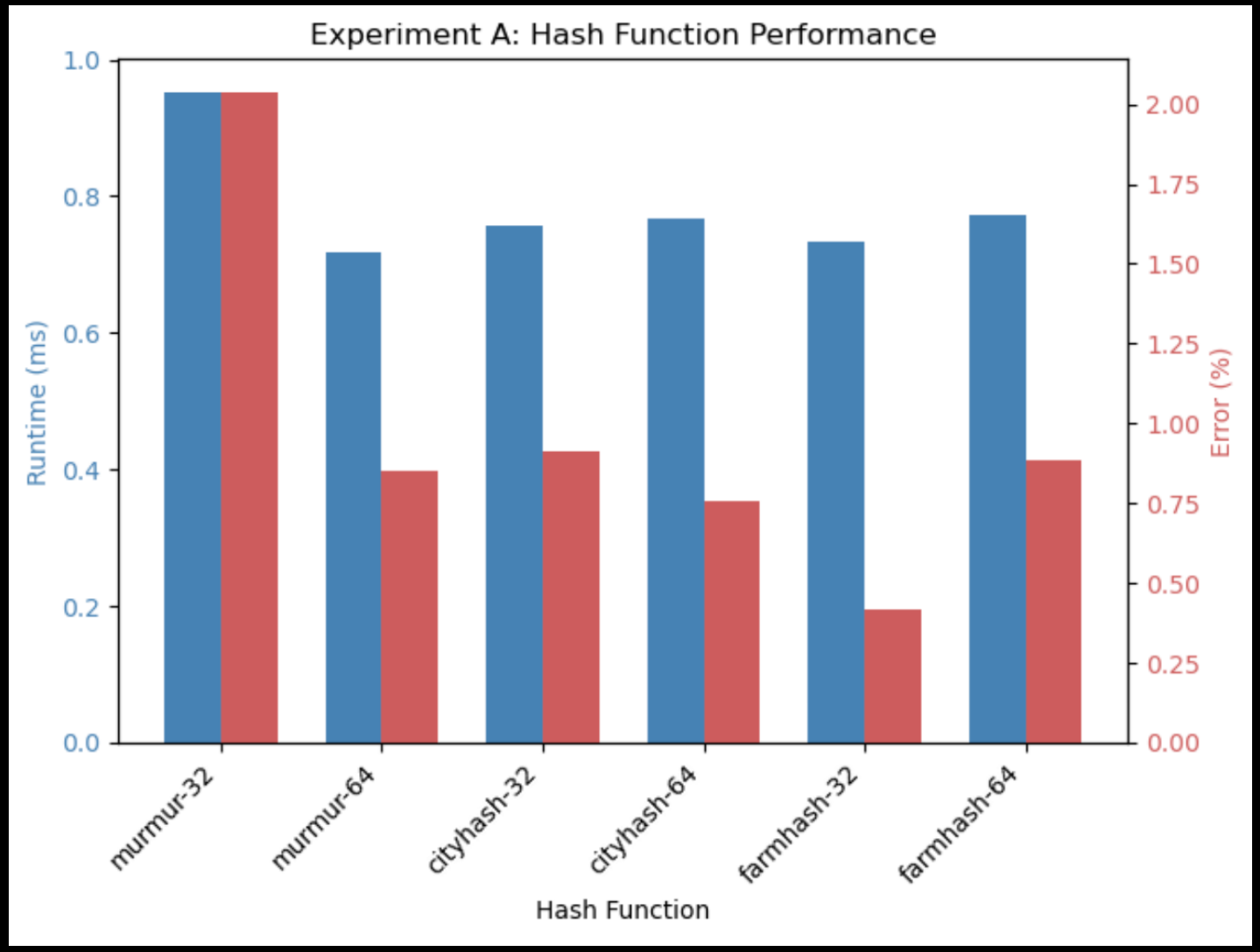
HASH FUNCTION IMPACT

1024 buckets (b=10)

Hash Function	Runtime	Error
murmur-32	0.933 ms	2.039%
murmur-64	0.703 ms	0.850%
cityhash-32	0.746 ms	0.913%
cityhash-64	0.743 ms	0.758%
farmhash-32	**0.722 ms**	**0.417%**
farmhash-64	0.755 ms	0.885%

Best hash function: farmhash-32

Hash Function Runtime and Accuracy Comparison



Experiment B

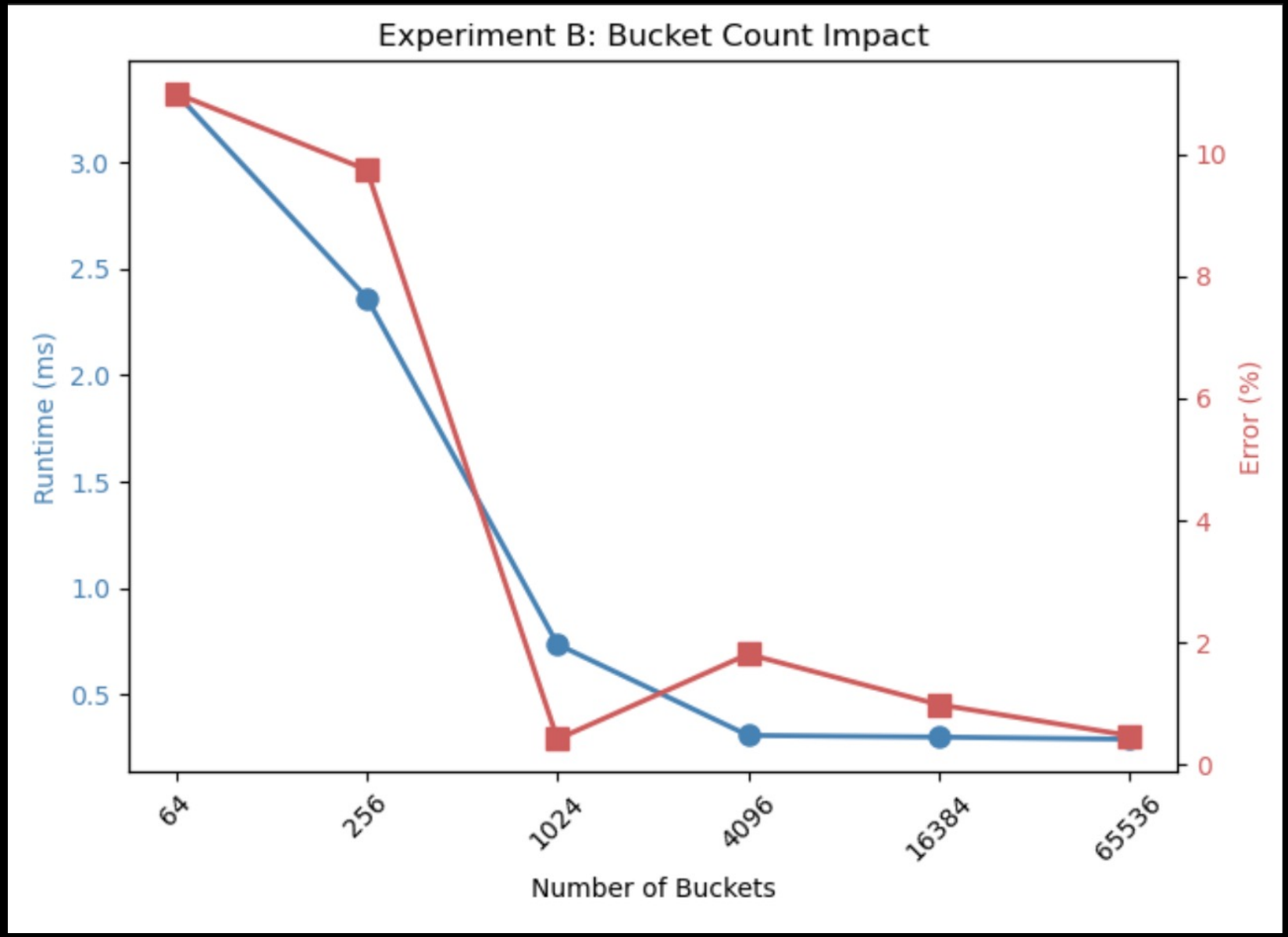
BUCKET COUNT IMPACT

Hash Function: farmhash-32

b	Buckets (m)	Runtime	Error %	Memory
6	64	3.259 ms	10.996%	0.25 KB
8	256	2.323 ms	9.744%	1.00 KB
10	**1024**	**0.721 ms**	**0.417%**	**4.00 KB**
12	4096	0.303 ms	1.809%	16.00 KB
14	16384	0.281 ms	0.984%	64.00 KB
16	65536	0.285 ms	0.477%	256.00 KB

Most Accurate bucket count: 1024, b = 10

Bucket Count Impact on Runtime and Accuracy



Recommended Configuration

Hash Function: farmhash-32

Bucket Count: 1024 (b=10)

Expected Error: 0.417%

Expected Runtime: 0.721 ms

Memory Usage: 4.00 KB

Task 2 CUDA Hash Table with OpenAddressing

Overview

Building a High-Performance Hash Table.

Dataset for this experiment:

Transactional_retail.csv

Hash: farmhash 32 bit

What is a Hash Table

Data structure for rapid data retrieval

Similar to a Python dictionary

Stores data as key → value pairs

O(1) average lookup

Open Addressing

When two keys are mapped to the same index there is collision.

Open Addressing solution: **Search for next available slot**

Why Open Addressing?

No Pointers used -> GPU Friendly

Open Addressing Methods

1. Linear Probing

Check next slot: index, index+1, index+2, index+3 ...

Simple and cache friendly

Problem: Creates long clusters of filled slots (slow system)

2. Quadratic Probing

Check slots: index, index+1^2, index+2^2, index+3^2 ...

Reduces clustering

Potentially better performance as table fills up

Index calculation for Hash Table

Traditional method:

index = hash % size -> slow

Bitwise method:

index = hash & (size - 1) -> fast

Needs the table size to be a power of 2

Experiment C
Linear Probing

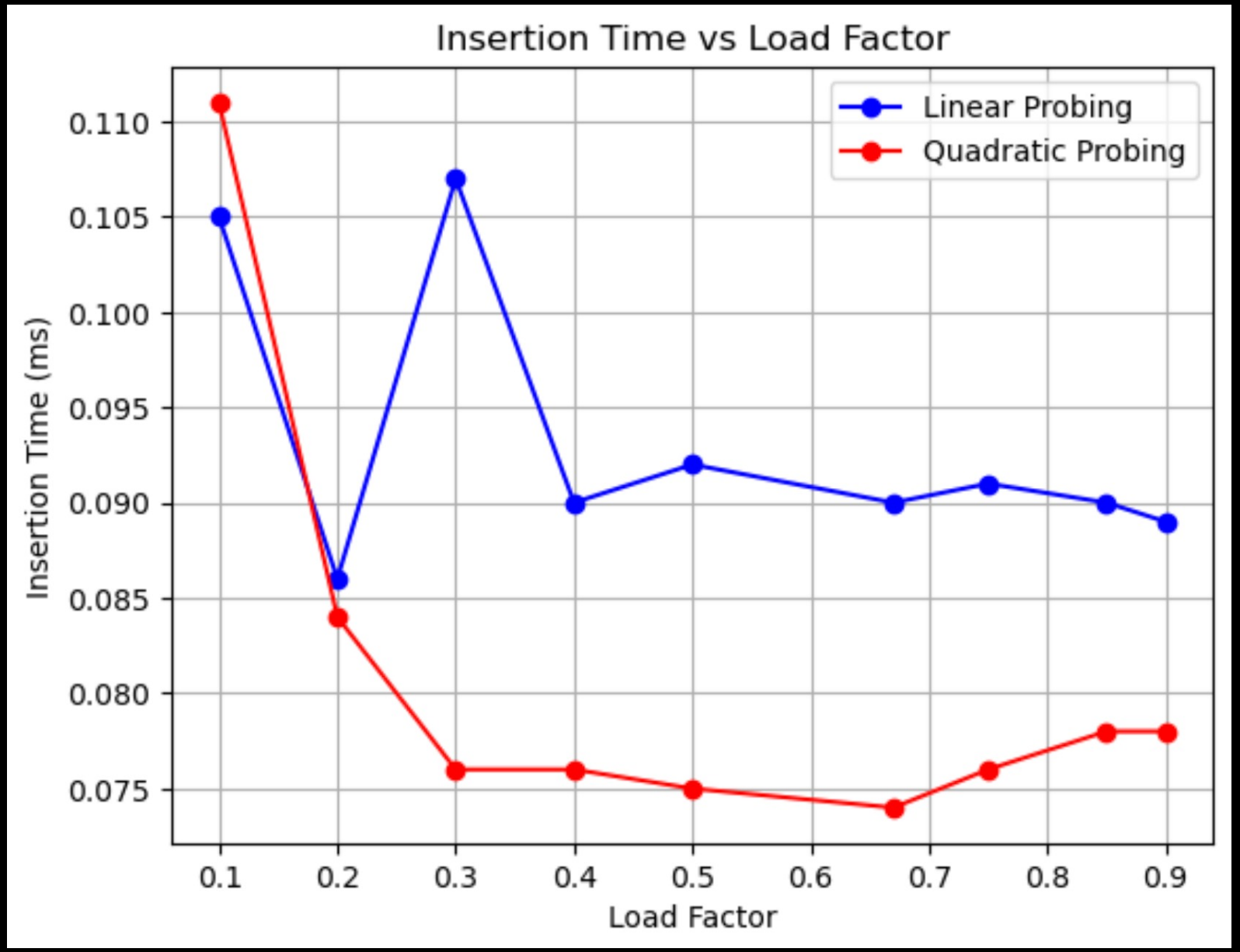
Load Factor	Table Size	Actual Load	Insertion Time (ms)	Memory (MB)
0.10	262,144	0.063	0.105	4.0
0.20	131,072	0.126	0.086	2.0
0.30	65,536	0.251	0.087	1.0
0.40	65,536	0.251	0.090	1.0
0.50	65,536	0.251	0.092	1.0
0.67	32,768	0.503	0.090	0.5
0.75	32,768	0.503	0.091	0.5
0.85	32,768	0.503	0.090	0.5
0.90	32,768	0.503	0.089	0.5

0.67 Load Factor is 1.5x sizing your table size

Experiment C
Quadratic Probing

Load Factor	Table Size	Actual Load	Insertion Time (ms)	Memory (MB)
0.10	262,144	0.063	0.111	4.0
0.20	131,072	0.126	0.084	2.0
0.30	65,536	0.251	0.075	1.0
0.40	65,536	0.251	0.074	1.0
0.50	65,536	0.251	0.074	1.0
0.67	32,768	0.503	0.076	0.5
0.75	32,768	0.503	0.076	0.5
0.85	32,768	0.503	0.078	0.5
0.90	32,768	0.503	0.078	0.5

0.67 and 0.75 uses half the memory for similar time


Insertion Time vs Load Factor
Linear Probing
Quadratic Probing
Insertion Time (ms)
0.110
0.105
0.100
0.095
0.090
0.085
0.080
0.075
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
Load Factor

Experiment C
Lookup Time

Method	Load Factor	Insertion Time (ms)	Lookup Time (ms)	Total Time (ms)
Linear	0.67	0.083	0.021	0.103
Quadratic	0.67	0.076	0.019	0.095

Quadratic Probing with 0.67 load factor is the best method for Hash Table

Hash Table Creation

We create the hash table for each item with the value part empty for future uses

item 1: [Trans 1: null, Trans 2: null]

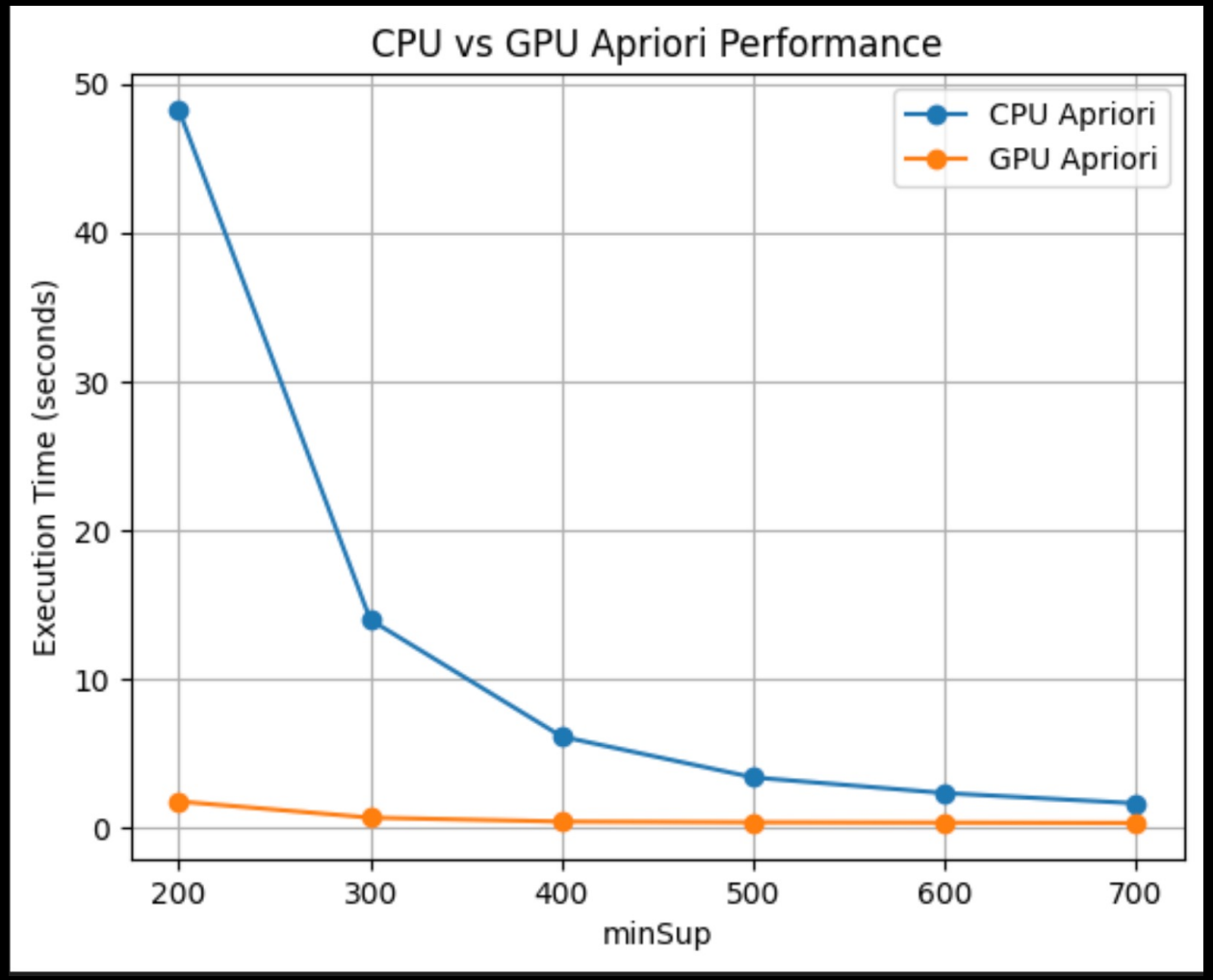
Item 2: [Trans 1: null, Trans 3: null]

Item 1: {1: null, 163: null, 229: null, 958: null, ...}

Item 2: {1: null, 66: null, 360: null, 439: null, ...}

Comparing Apriori on GPU vs CPU

minSup	CPU Apriori (s)	GPU Apriori (s)
700	1.6354	0.3159
600	2.3224	0.3392
500	3.3756	0.3622
400	6.1183	0.4082
300	13.9712	0.6732
200	48.2433	1.7585



Summary

HyperLogLog

farmhash-32, b=10: 0.417% measured error, 4.00 KB

CUDA hash table

Quadratic probing, requested load factor 0.67

0.095 ms total vs 0.103 ms for linear in the lookup experiment

32,768 slots, actual load 0.503, 0.5 MB

GPU Apriori

5.18x to 27.43x faster across the reported minSup settings

At minSup=200: 48.2433 s on CPU vs 1.7585 s on GPU